- Term Project Check-in #2 next week
  - Details and links to Calendy will be posted under Announcements on myCourses after class
  - Remember each checkin is 5% of your project grade
  - Check-in details can be found on myCourses at Content > Project > Project Check-in

Mid-term Exam Class Statistics

Number of submitted grades: 19 / 19

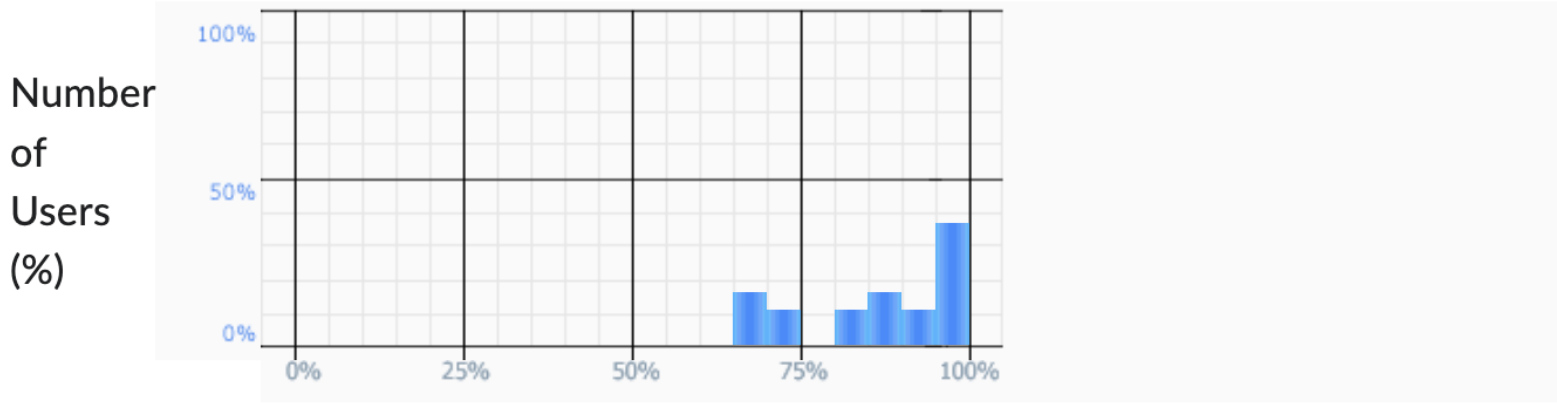
Minimum:		D (66 %)
Maximum:		A (98 %)
Average:		B (86.21 %)

Mode: A (96 %)

Median: B+ (88 %)

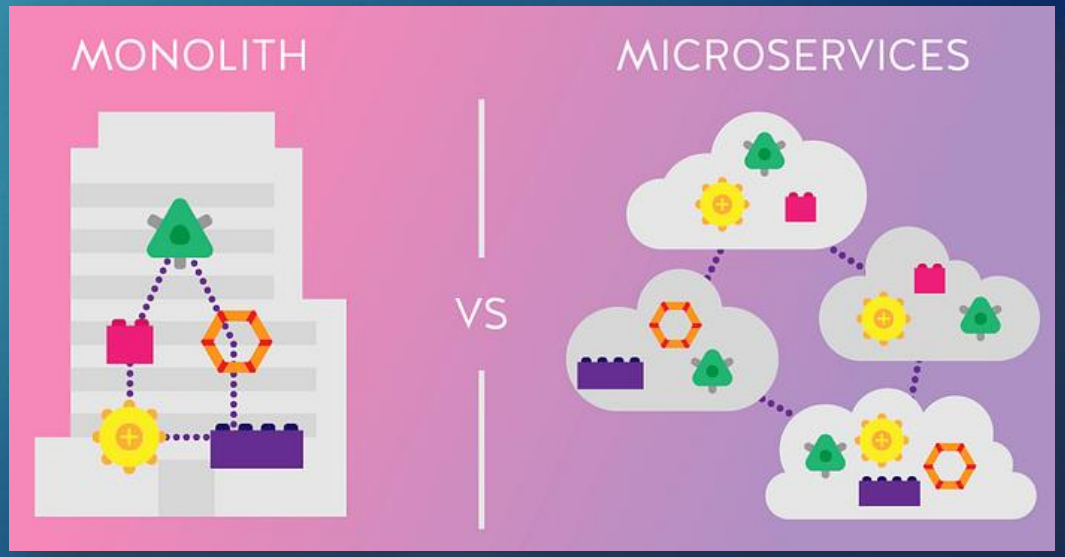
Standard Deviation: F (11.33 %)

Grade Distribution



RECAP

- Monolithic Architecture
  - Unified, tightly-coupled application design
  - Components depend on each other (UI, logic, data)
  - Hard to scale, maintain, and deploy
  - Not ideal for complex or rapidly evolving systems
- Microservices Architecture
  - Decoupled, modular services focused on specific business tasks
  - Enables independent development, deployment, and scaling
  - Promotes agility, reuse, and fault isolation
  - Best deployed with orchestration (e.g., Kubernetes, ECS)



Source: https://19yw4b240vb03ws8qm25h366-wpengine.netdna-ssl.com/wp-content/uploads/Monolith-vs-Microservices.png

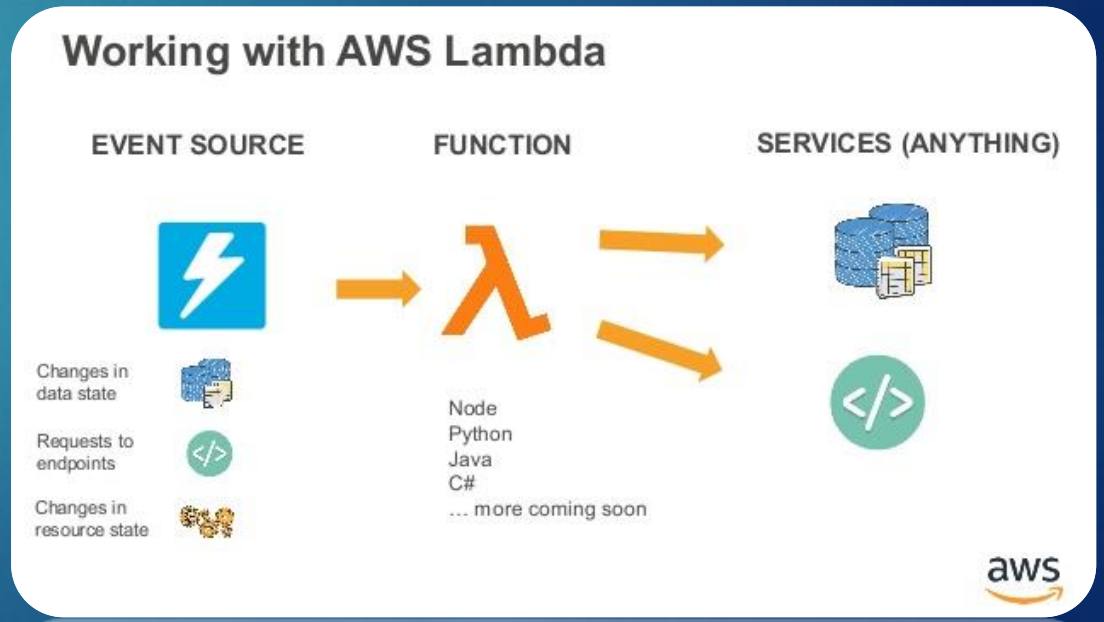
RECAP

- Microservice Communication
  - RESTful APIs for synchronous calls
  - Message Queues (e.g., Amazon SQS) for asynchronous communication
  - Async is better for scalability & resilience
- Serverless Computing
  - Pay-per-use model (e.g., AWS Lambda, AWS Fargate)
  - No server management required
  - Event-driven and auto-scaling
  - Common triggers: API Gateway calls, file uploads (S3), queue messages

Source: https://www.techmagic.co/blog/serverless-vs-microservices-which-architecture-to-choose

RECAP

- AWS Lambda
  - Automatically runs backend code in response to events
  - Supports many languages (Python, Java, Node.js, etc.)
  - Invoked by S3, API Gateway, queues, etc.
  - Pricing based on requests & duration
- Pros:
  - Cost-effective, scalable, fast
- Considerations:
  - Debugging & testing can be harder
  - Resource/time limits
  - Vendor lock-in risk



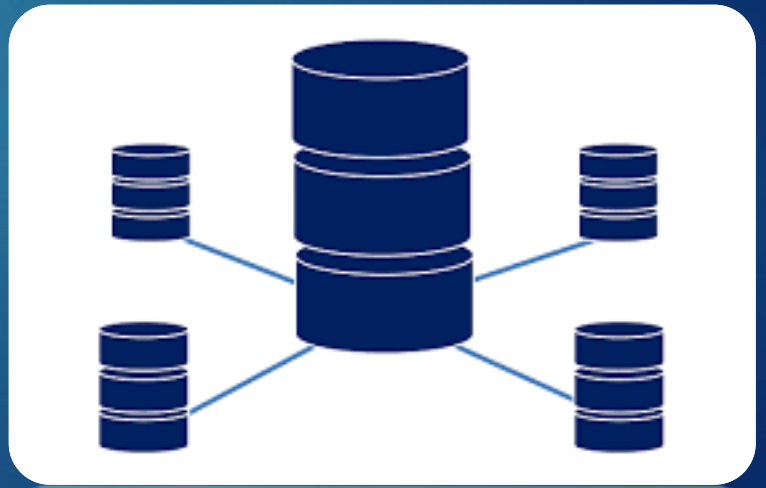
Databases in the Cloud (SQL vs. NoSQL)

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology

Relational Databases

- Relational databases are based on the relational model, an intuitive, straightforward way of representing data in tables
- Each row in the table is a record with a unique ID called the key
- The standard user and application programming interface of a relational database is the Structured Query Language (SQL)
- If you want to use a relational (aka SQL) database on AWS, you have two options:
  1. Operate a relational database yourself on top of virtual machines, which we did with our WordPress example using MySQL
  2. Use the Amazon Relational Database Service (RDS)



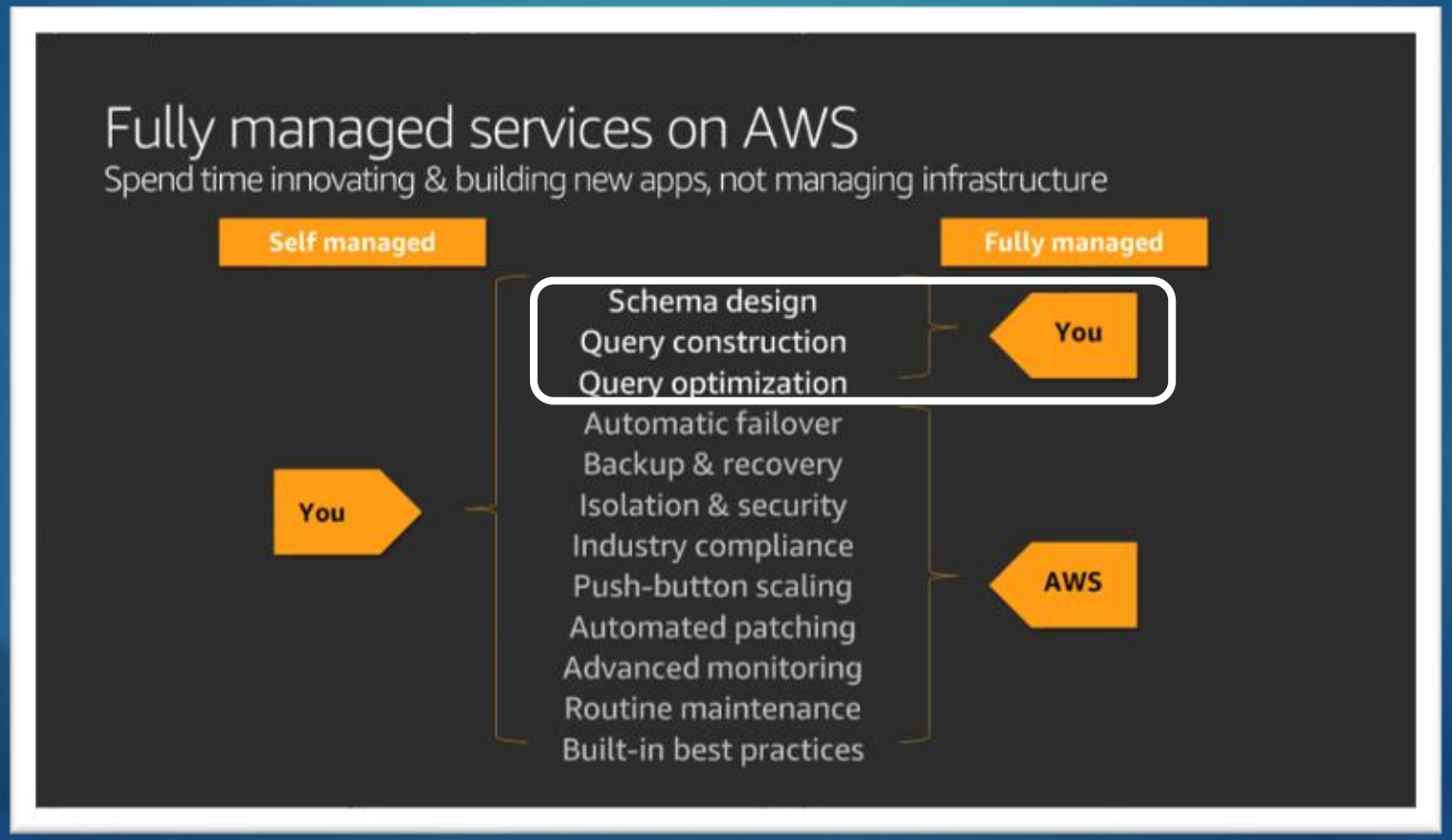
Amazon Relational Database Service (RDS)

- Amazon RDS is a managed database service
- It is not a database, but instead provides multiple database products to choose including Aurora, PostgreSQL, MySQL, MariaDB, Oracle, and Microsoft SQL Server

- It is also commonly referred to as Database as a Service (DBaaS)

Managed Database Service

- What does this mean for what you would manage vs. the cloud provider (AWS?)

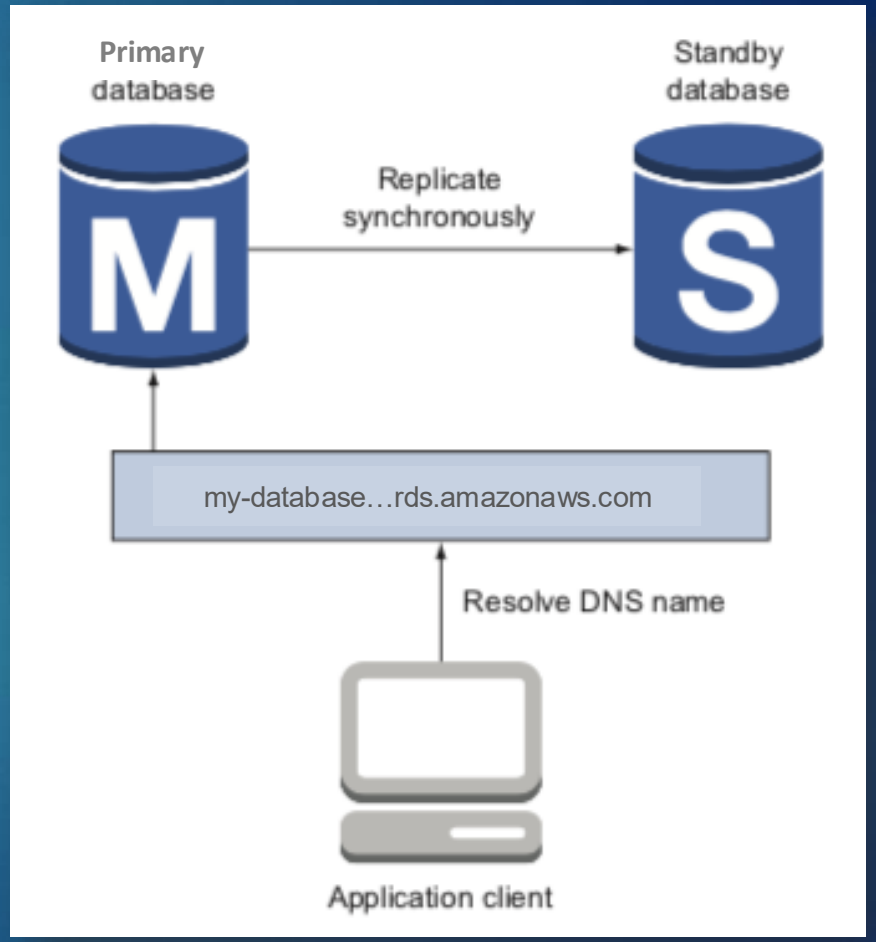


Amazon RDS - Features & Benefits

- **Managed Database Administrative Tasks** – RDS automates administrative tasks such as database backups, software patching, failure detection, and recovery
- **Scalability** – Scale vertically by increasing CPU, memory, or storage capacity, or horizontally by adding read replicas to offload read traffic and improve performance
- **Security** - provides various security features, including encryption at rest, in transit and network isolation via VPC
- **High Availability** – RDS offers a feature called Multi-AZ, which provides an SLA up-time of 99.95%. AWS provides a synchronous "standby" replica of every database in another "zone."
  - Since both the database and its replica are in sync, there is no chance of data loss

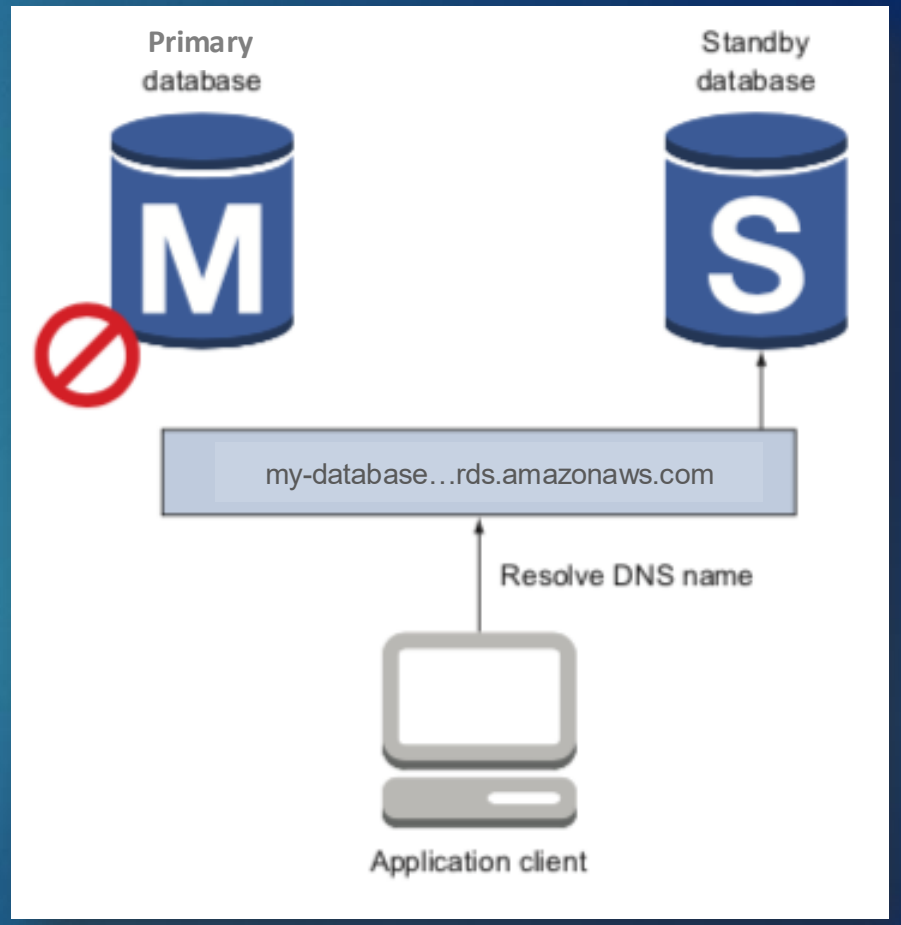
Amazon RDS – High Availability

- Amazon RDS lets you launch highly available (HA) databases
- Compared to a default database consisting of a single database instance, an HA RDS database consists of two database instances: a primary and a standby database
- All clients send requests to the primary database
- Data is replicated between the primary and the standby database synchronously



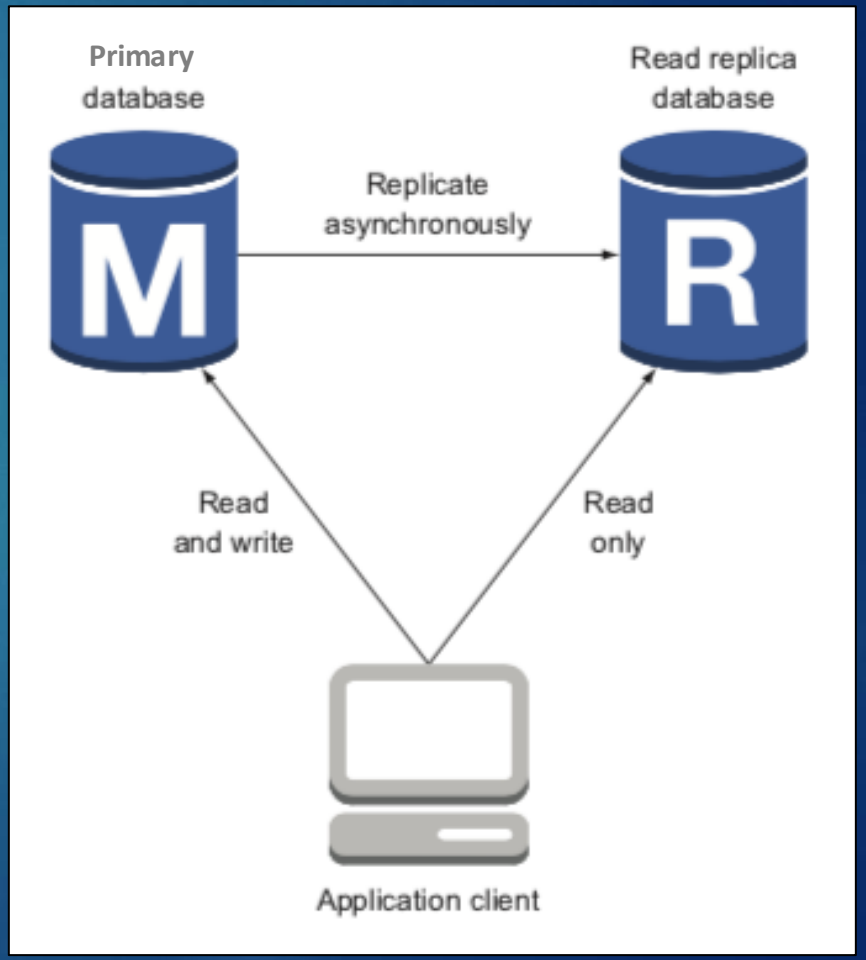
Amazon RDS – High Availability

- If the primary database becomes unavailable due to hardware or network failures, RDS starts the failover process
- RDS automatically performs a failover to the standby so that database operations can resume quickly without administrative intervention
- The standby database then becomes the primary database



Amazon RDS – Read Replication

- A database suffering from too many read requests can be scaled horizontally by adding additional database instances for read replication
- Changes to the database are asynchronously replicated to an additional read-only database instance
- The read requests can be distributed between the primary database and its read-replication databases to increase read throughput



Aurora vs other DB offerings…

- Better performance
- Compatible with MySQL and PostgreSQL
- Serverless option
  - Database will automatically start up, shut down, and scale capacity up or down based on your application's needs
- Native High Availability including "multi-primary"
  - You can multiple read/write instances of your database across multiple AZ
- Downsides
  - Can be more expensive, vendor lock-in

Amazon RDS (managed) vs. Self-hosting

- You would need considerable time and staff know-how to build a comparable relational database environment based on virtual machines

	Amazon RDS	Self-hosted on virtual machines
Cost for AWS services	Higher because RDS costs more than virtual machines (EC2)	Lower because virtual machines (EC2) are cheaper than RDS
Total cost of ownership	Lower because operating costs are split among many customers	Much higher because you need your own manpower to manage your database
Quality	AWS professionals are responsible for the managed service.	You'll need to build a team of professionals and implement quality control yourself.
Flexibility	High, because you can choose a relational database system and most of the configuration parameters	Higher, because you can control every part of the relational database system you installed on virtual machines

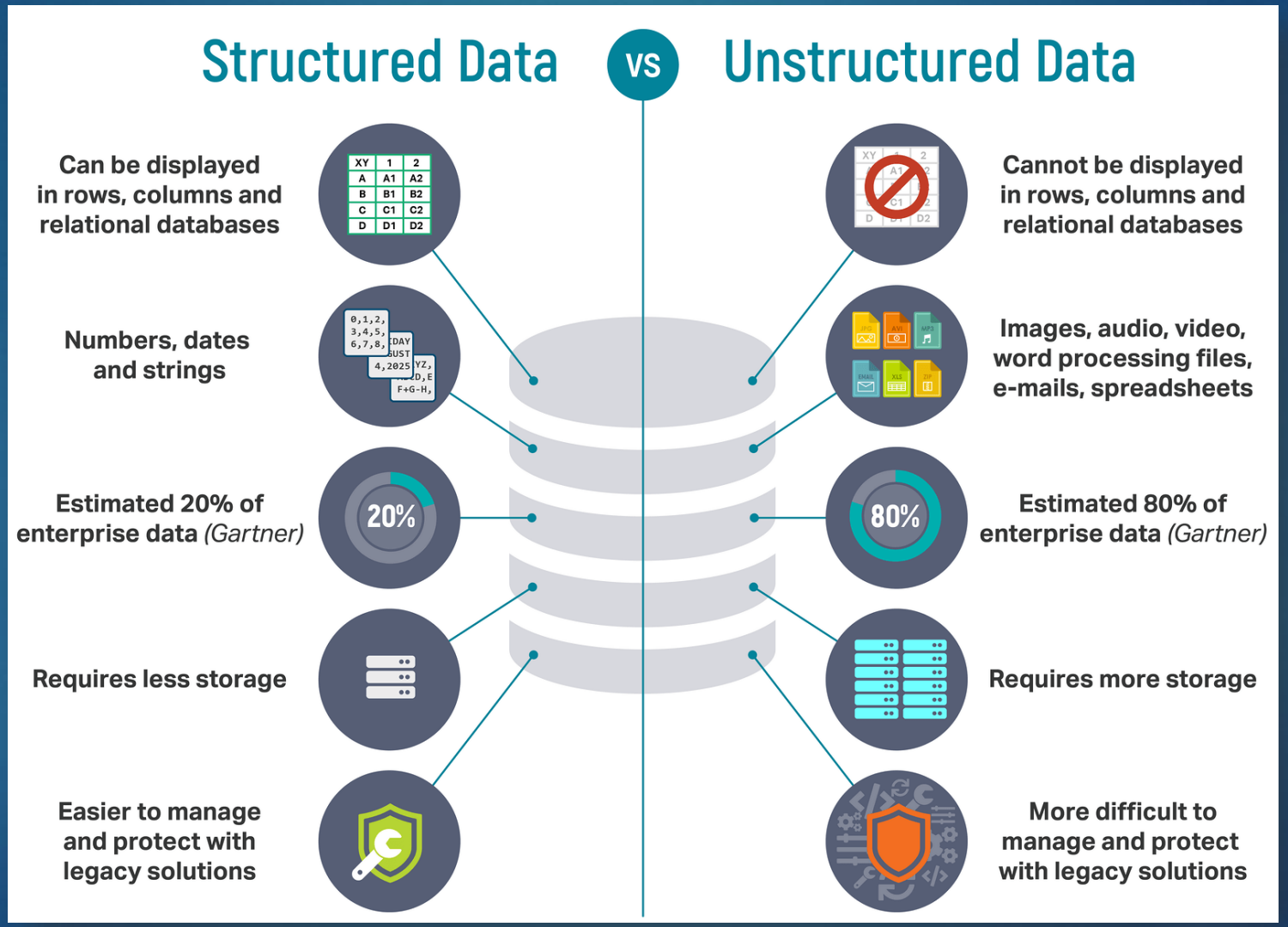
SQL Databases - Recap

- If your data is primarily structured, a SQL database is likely the right choice
- A SQL database is a great fit for transaction-oriented systems such as customer relationship management tools, accounting software, and e-commerce platforms
- SQL databases are best when you need ACID compliance
  - Atomicity ➔Each transaction either succeeds completely or is fully rolled back
  - Consistency ➔ Data written to a database must be valid according to all defined rules
  - Isolation ➔ When transactions are run concurrently, they do not contend with each other, and act as if they were being run sequentially
  - Durability ➔ Once a transaction has been committed to the database, it is considered permanent, even in the event of a system failure

SQL Databases - Challenges

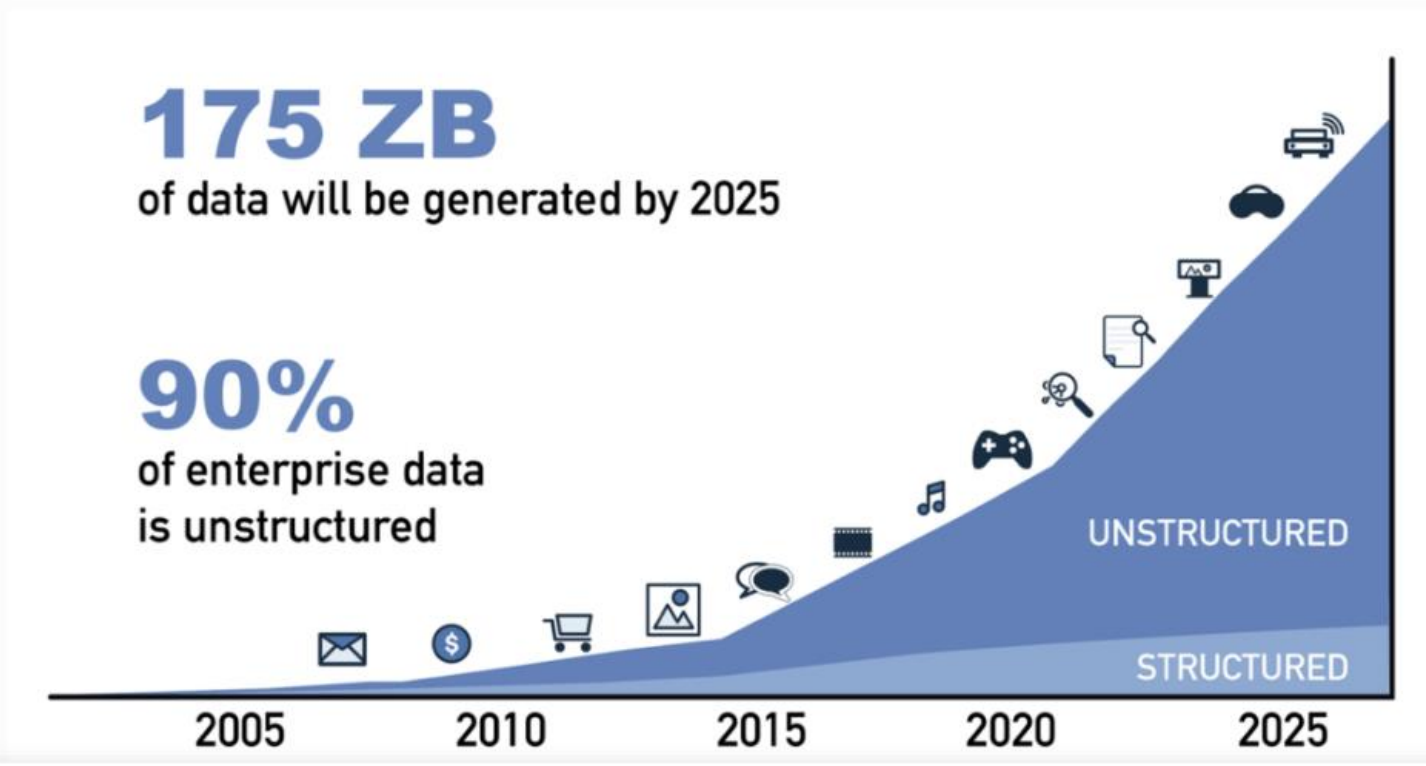
- With SQL databases, there is overhead for complex select, update and deletes:
  - Select – Joining too many tables to create a huge table
  - Update – Each update affects other tables
  - Delete – Must guarantee consistency of data
- They can be costly to scale
  - Generally, they scale vertically vs. horizontally
- SQL databases don't do well with unstructured data or scaling with extremely high volumes of data

SQL Databases - Challenges



Source: https://lawtomated.com/structured-data-vs-unstructured-data-what-are-they-and-why-care/

SQL Databases - Challenges



In one day:
24 million transactions processed by Walmart
100 TB of data uploaded to Facebook
175 million tweets on Twitter (X)

How do you store, query and process this data efficiently?

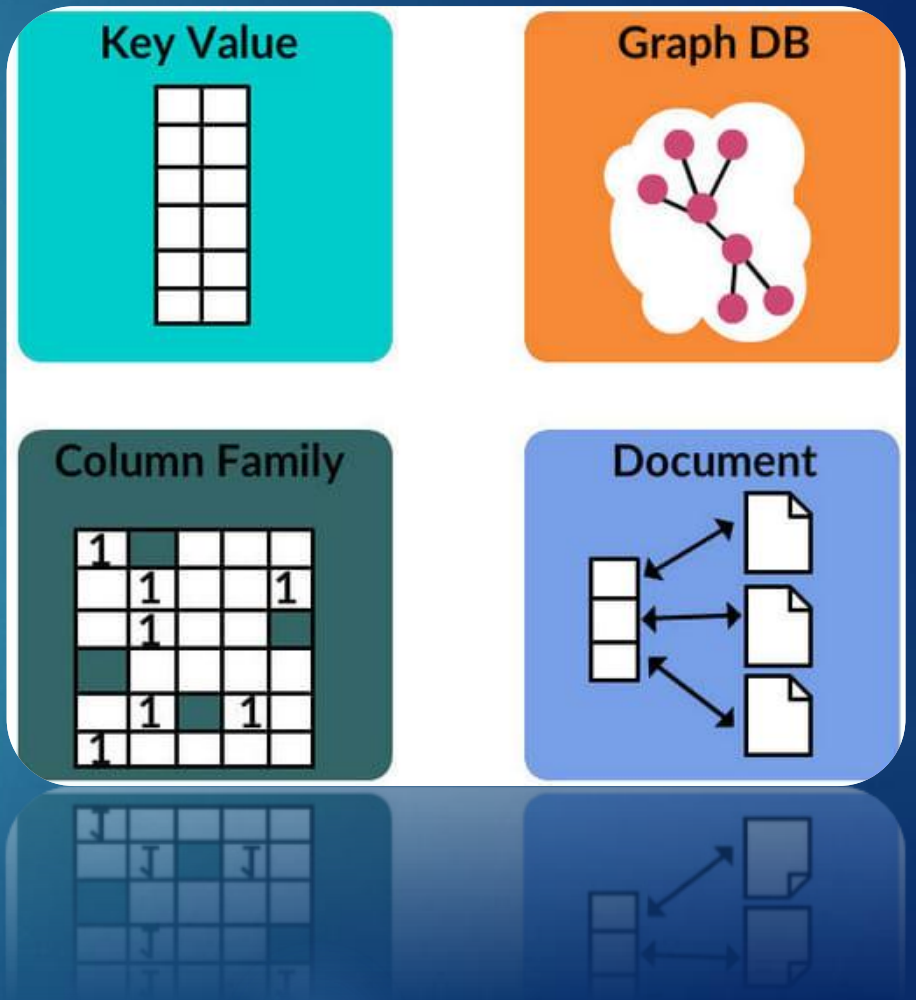
Answer: NoSQL

1 Zetabyte (ZB) ➔1million petabytes ➔ 1billion terabytes ➔1trillion gigabytes

Source: https://superna.io/resources/unleashing-power-unstructured-data

What is NoSQL?

- NoSQL is an approach to databases that represents a shift away from traditional relational database management system
- They do not rely on tables, columns, rows or schemas and use more flexible data models
- NoSQL follow "schema on read" vs. "schema on write" compared to SQL databases
- They are widely recognized for their ease of development, functionality, and performance at scale
- NoSQL databases use a variety of data models, including key/value, graph, column stores and document



NoSQL DBs – Key-Value

- Key-Value are the simplest NoSQL databases
- Every single item in the database is stored as an attribute name (or 'key'), together with its ‘value’
- A values can be any type of binary object (text, video, JSON document, etc.)
- Examples: Berkeley DB, Cassandra and AWS DynamoDB

Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

Key-Value DB - Example

SQL

Car

StateKey	CarKey	MakeKey	ModelKey	ColorKey	Year
1	1	1	1	2	2003
2	2	2	1	3	2005
3	3	2	1	2	2005

Color

ColorKey	Color
1	Red
2	Green
3	Blue

MakeModel

ModelKey	MakeKey	Model
1	1	Pathfinder
1	2	Bluebird
2	1	Civic

Make

MasterKey	Make
1	Nissan
2	Honda

State

StateKey	Name
1	NY
2	Maine
3	Florida

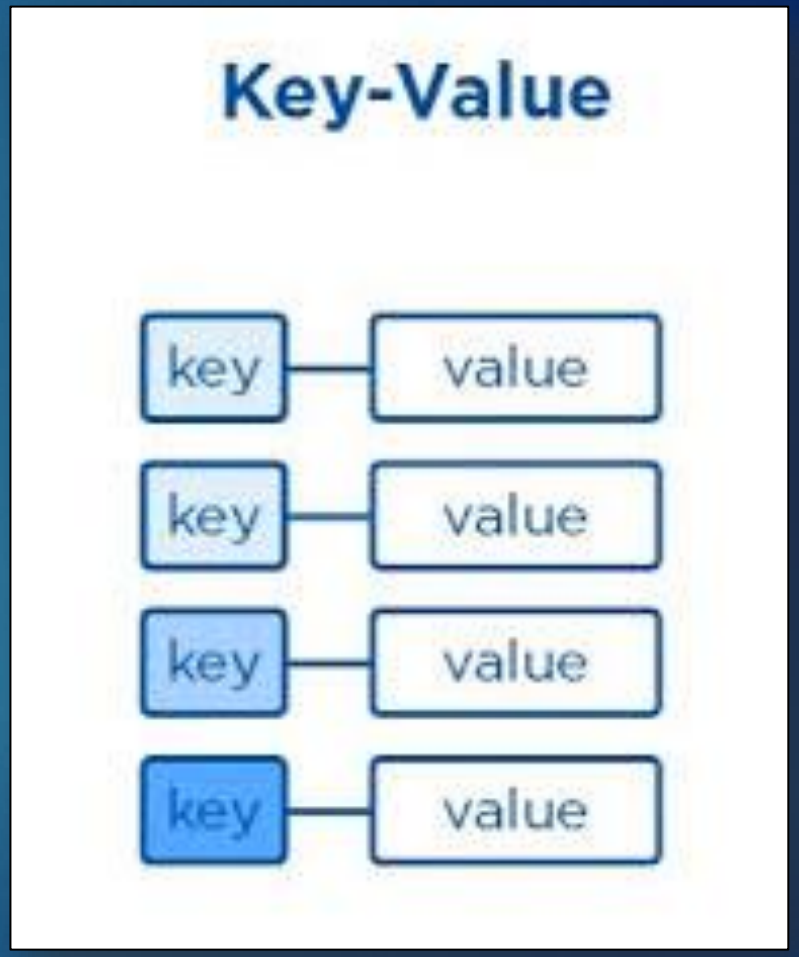
NoSQL

Key	Value
1	Make: Nissan Model: Pathfinder Color: Green Year: 2003 State: NY
2	Make: Honda Model: Civic Color: Green Year: 2005 State: Maine

- Question – What would I need to do to add a State value?

Key-Value - Benefits

- Simple data format makes write and read operations fast
- Both keys and values can be anything, ranging from simple objects to complex compound objects
- Key-value databases are highly partitionable and allow horizontal scaling at scales that other types of databases cannot achieve



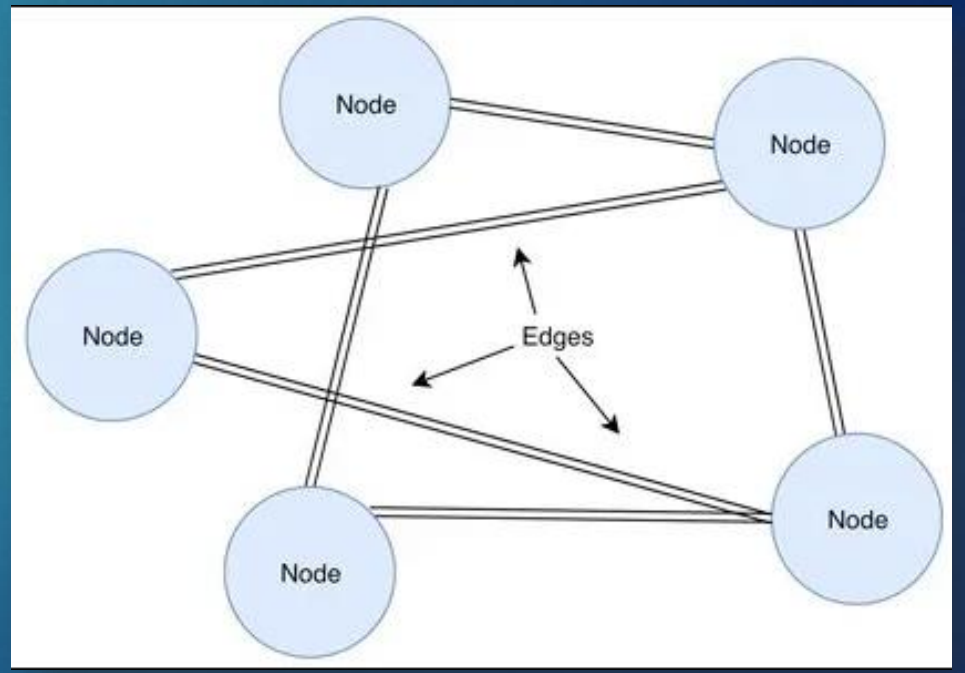
NoSQL DBs – Document

- Similar to key-value databases
- Used for storing, retrieving, and managing semi-structured data
- No single schema as they can contain many different key-value pairs, or key-array pairs, or even nested documents
- They typically store self-describing JSON, XML, and BSON (Binary JSON) documents
- Popular fields in the document can be indexed to provide fast retrieval without knowing the key
- Examples: MongoDB, Apache CouchDB and AWS DocumentDB

Key	Document
1001	{ "CustomerID": 99, "OrderItems": [{ "ProductID": 2010, "Quantity": 2, "Cost": 520 }, { "ProductID": 4365, "Quantity": 1, "Cost": 18 }], "OrderDate": "04/01/2017" }
1002	{ "CustomerID": 220, "OrderItems": [{ "ProductID": 1285, "Quantity": 1, "Cost": 120 }], "OrderDate": "05/08/2017" }

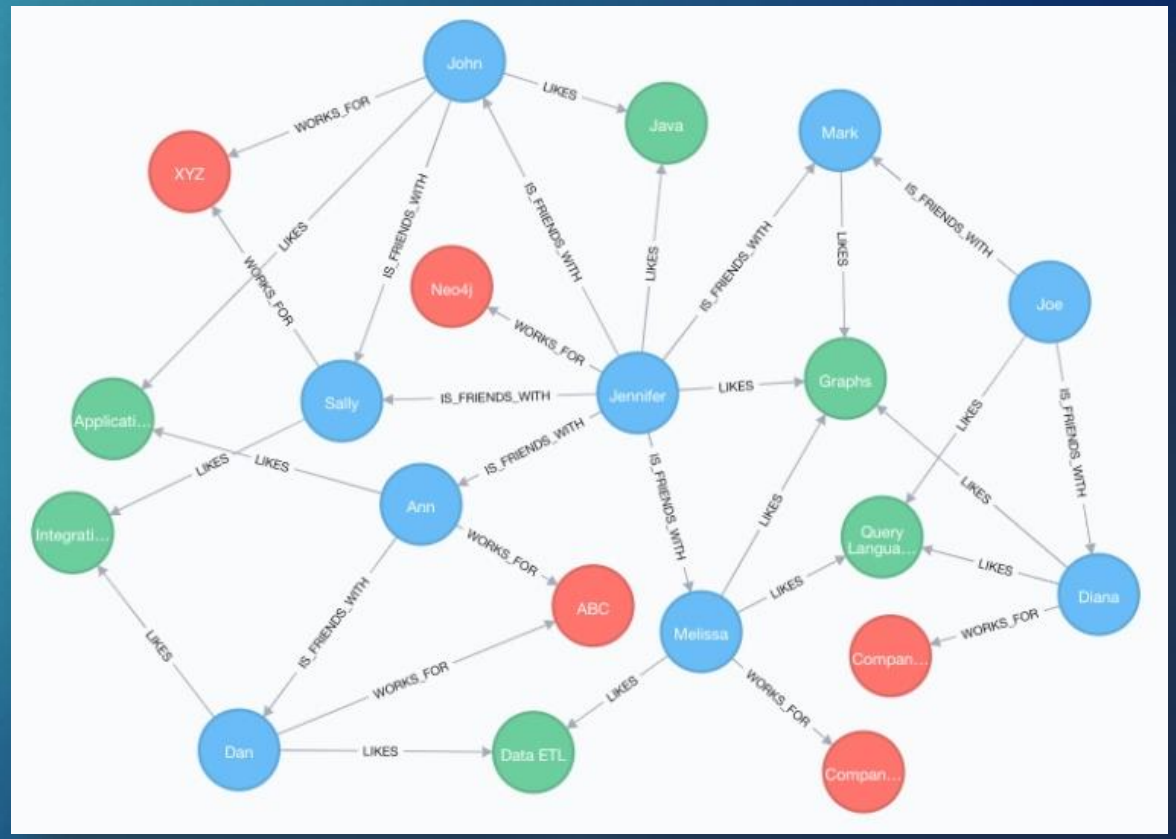
NoSQL DBs – Graph Database

- Graph databases are purpose-built to store and navigate relationships
- Graph databases use nodes to store data entities, and edges to store relationships between entities
- An edge always has a start node, end node, type, and direction, and an edge can describe parent-child relationships, actions, ownership, and the like
- There is no limit to the number and kind of relationships a node can have
- Examples: Neo4j, Microsoft CosmosDB and Amazon Neptune



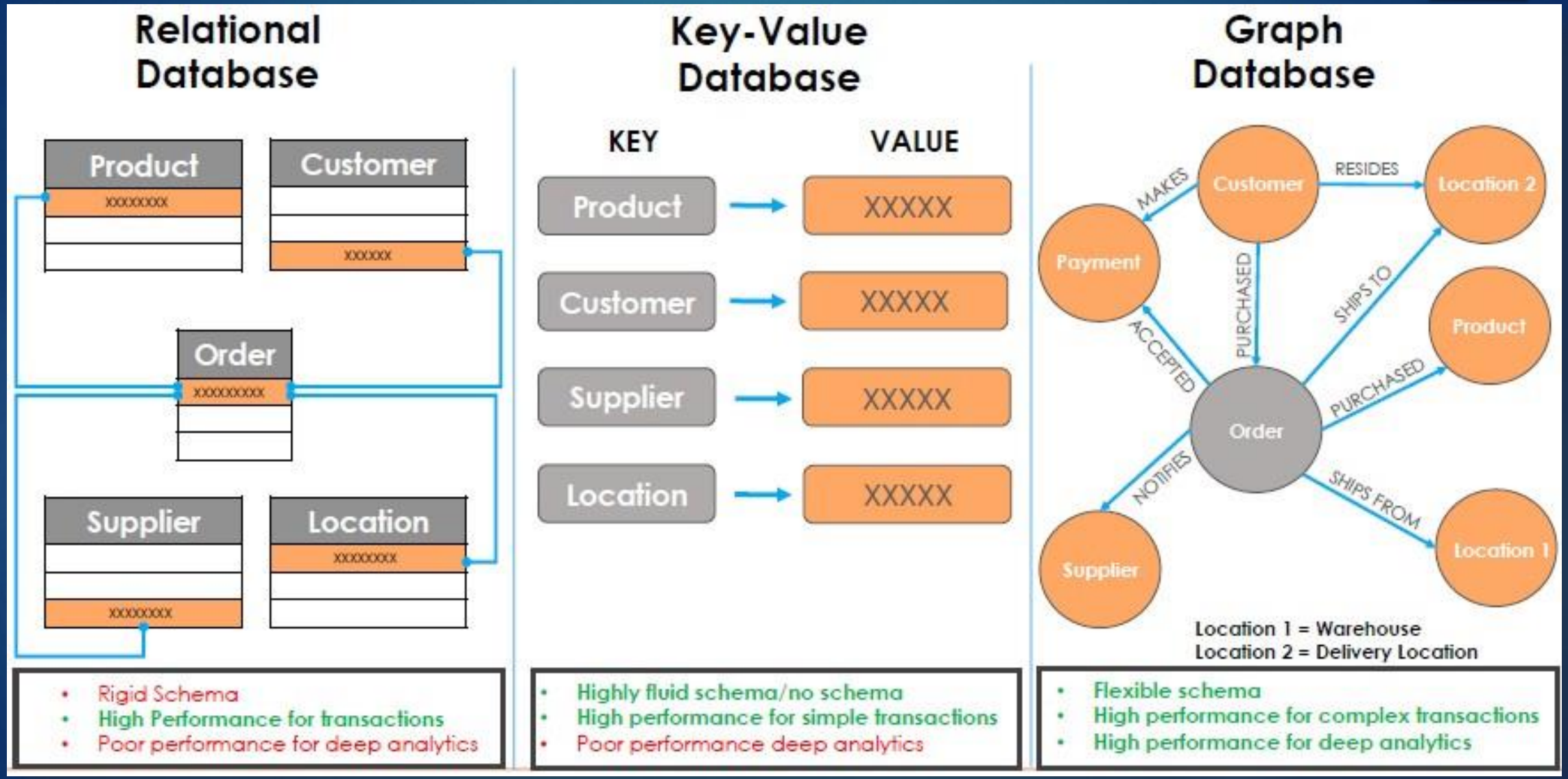
Graph Database – Benefits

- Can store and process large volumes of data and enable the search, discovery, and exploration of large networks
- Ideal for solutions such as Fraud Detection, 360 Customer views and Social Networks
- Query speed only dependent on the number of concrete relationships, and not on the amount of data
- Compliance to standards (RDF, SPARQL), no vendor lock-in
- Easy to publish/consume (e.g. Knowledge Graphs – more on this later)



Source: https://neo4j.com/

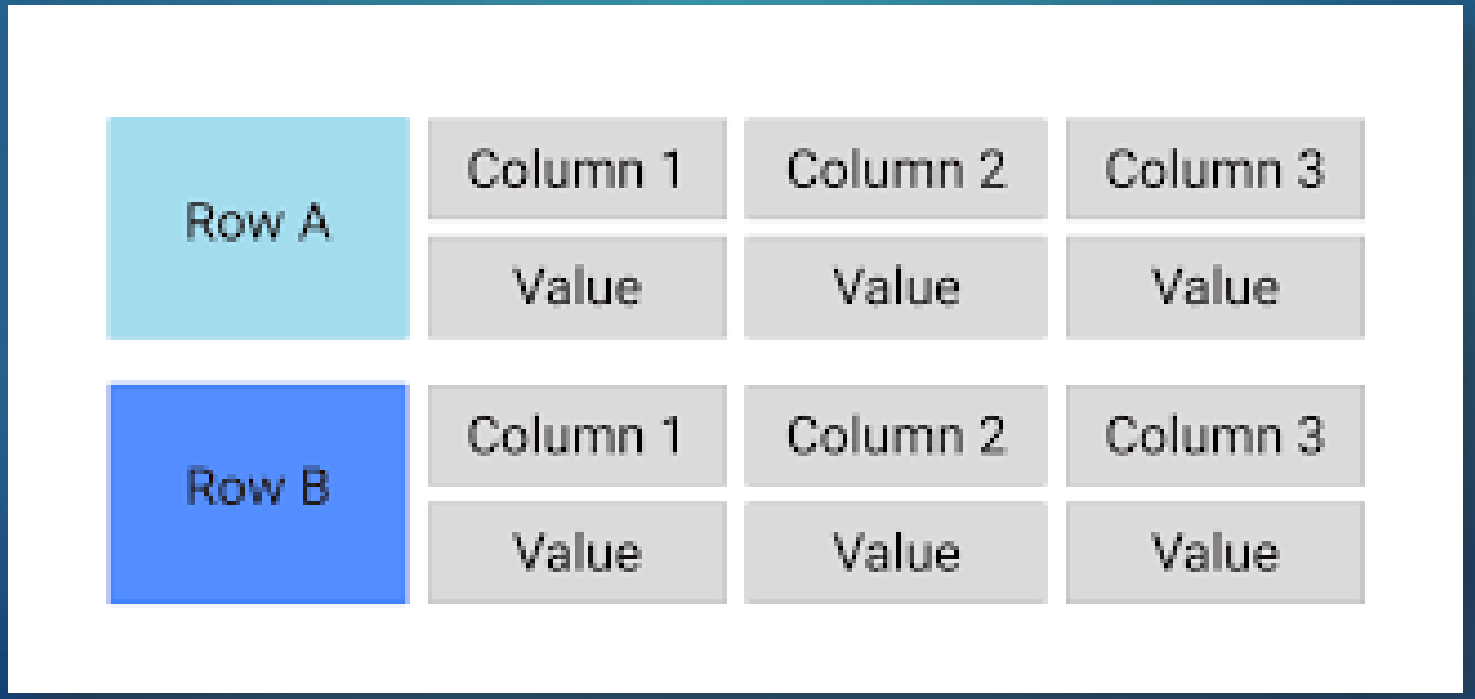
Relational vs. Key-Value vs. Graph Database



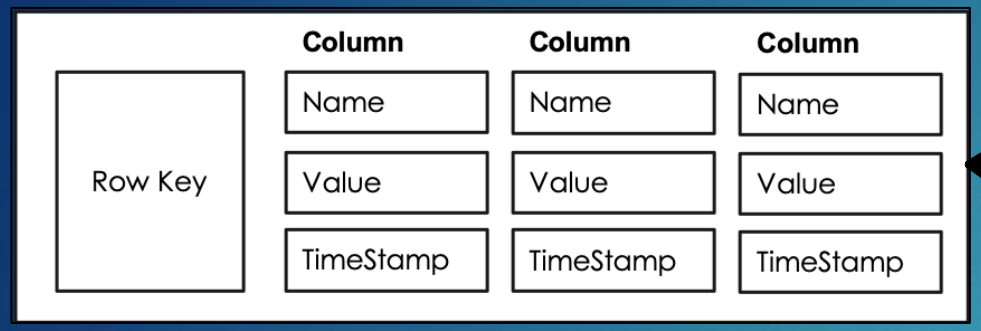
Source: https://www.nextplatform.com/2018/09/19/the-graph-database-poised-to-pounce-on-the-mainstream/

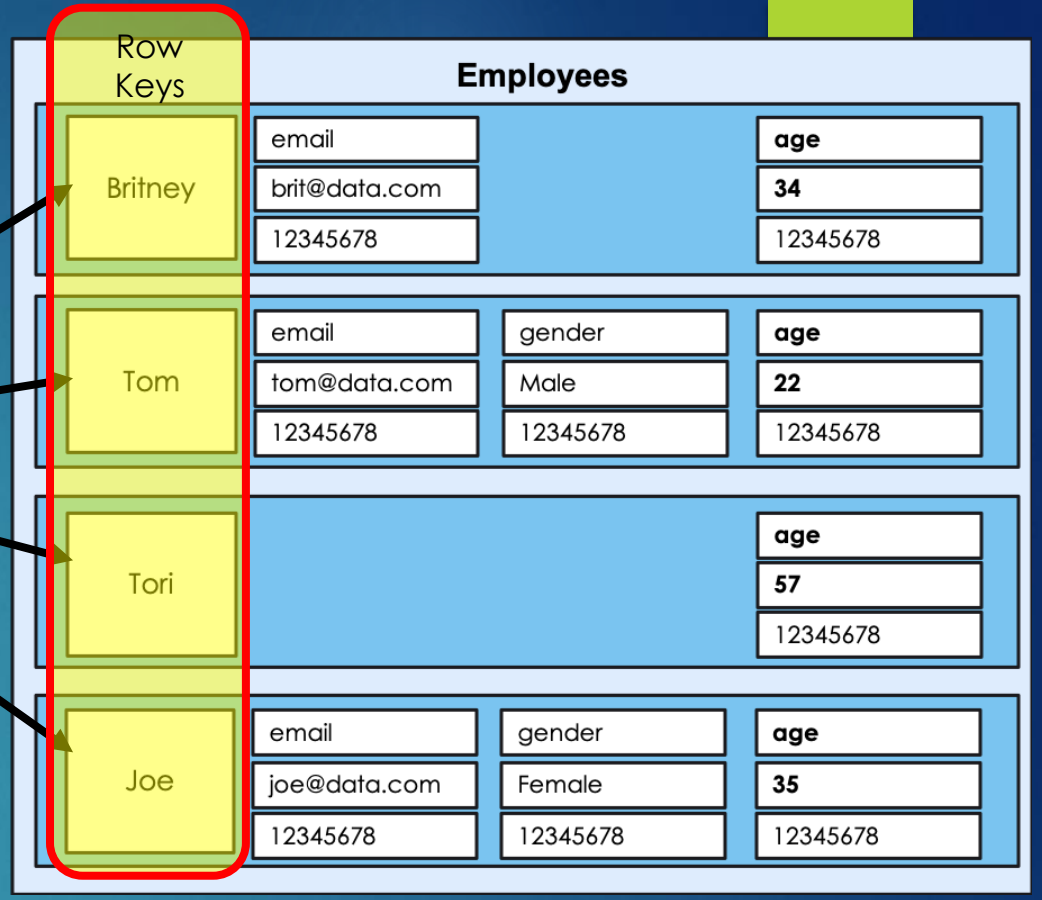
NoSQL DBs – Column Store

- NoSQL column stores appear similar to SQL tables, but they store data differently
- Data is arranged in column groups that keep related data together and can scale to include thousands or even millions of columns



Column Store DBs - Example





- **Row Key -** Each row has a unique key, which is a unique identifier for that row.
- **Column -** Each column contains a name, a value, and timestamp:
  - **Name -** This is the name of the name/value pair
  - **Value -** This is the value of the name/value pair
  - **Timestamp -** This provides the date and time that the data was inserted. This can be used to determine the most recent version of data.

- A **column family** consists of multiple rows (e.g. Employees)
- Each **row** can contain a different number of columns to the other rows. The columns don't have to match the columns in the other rows (i.e. they can have different column names, data types, etc).
- Each **column** is contained to its row and doesn't span all rows like in a relational database. It contains a name/value pair, along with a timestamp.

Column Store DBs - Benefits

- **Compression -** Column stores are very efficient at data compression and/or partitioning
- **Aggregation queries -** Due to their structure, columnar databases perform particularly well with aggregation queries (such as SUM, COUNT, AVG, etc)
- **Scalability –** Column store databases are very scalable. They are well suited to massively parallel processing (MPP), which involves having data spread across a large cluster of machines – often thousands of machines
- **Fast to load and query -** Columnar stores can be loaded extremely fast. A billion-row table could be loaded within a few seconds. You can start querying and analyzing almost immediately.
- Examples: Google Bigtable, HBase & Cassandra

NoSQL DBs in AWS - DynamoDB

- DynamoDB is Key-value and Document database that can delivers single-digit millisecond performance at any scale
- It's a fully managed, multi-region, multi-master, durable database with built-in security, backup and restore, and in-memory caching for internet-scale applications
- There are no servers to provision, patch, or manage, and no software to install, maintain, or operate. It automatically scales tables to adjust for capacity and maintains performance with zero administration
- Highly scalability as it can handle more than 10 trillion requests per day and can support peaks of more than 20 million requests per second
- Supports encryption at rest

DynamoDB – Features

- Data types:
  - **Single-valued** – Number, String, Binary, Boolean, and Null
  - **Multi-valued** – String Set, Number Set, and Binary Set
  - **Document** – List and Map
- Supports both Query and Scan for DB searching
  - Query
    - Faster as it searches only primary key attribute values and supports a subset of comparison operators on key attribute values to refine the search process
  - Scan
    - Less efficient as it scans the entire table
    - You can specify filters to apply to the results to refine the values returned to you, after the complete scan
- Class activity will look at these further

NoSQL – Recap

- NoSQL provides high level of scalability
- Able to handle large volumes of structured, semi-structured and unstructured data
- Schema on read (NoSQL) vs. schema on write (relational)
- It is used in distributed computing environment
- Implementation is less costly
- Programming is easy to use and flexible

NoSQL – Challenges

Data Consistency

- Most NoSQL databases do not perform ACID transactions for ensuring that data remains consistent across the entire database as it is moved around
- NoSQL relies on the principle of "eventual consistency", and it poses the risk that data on one database node may go out of sync with data on another node

Lack of Standards

- The design and query languages of NoSQL databases vary widely between different NoSQL products

Less Support

- Many NoSQL systems are open-source projects and companies offering support are often are small and/or start-ups without the global reach, support resources, or credibility of an Oracle, Microsoft, or IBM

When to pick SQL vs. NoSQL?

SQL more ideal for:

- **Transactions & Data Consistency** - Anything to do with money or numbers that requires transactions, ACID and data consistency will be very important giving clear advantage to Relational Databases
- **Storing Relationships** - Relational databases are built to store relationships. They have been tried & tested & are used by big guns in the industry like Facebook.

NoSQL more ideal for:

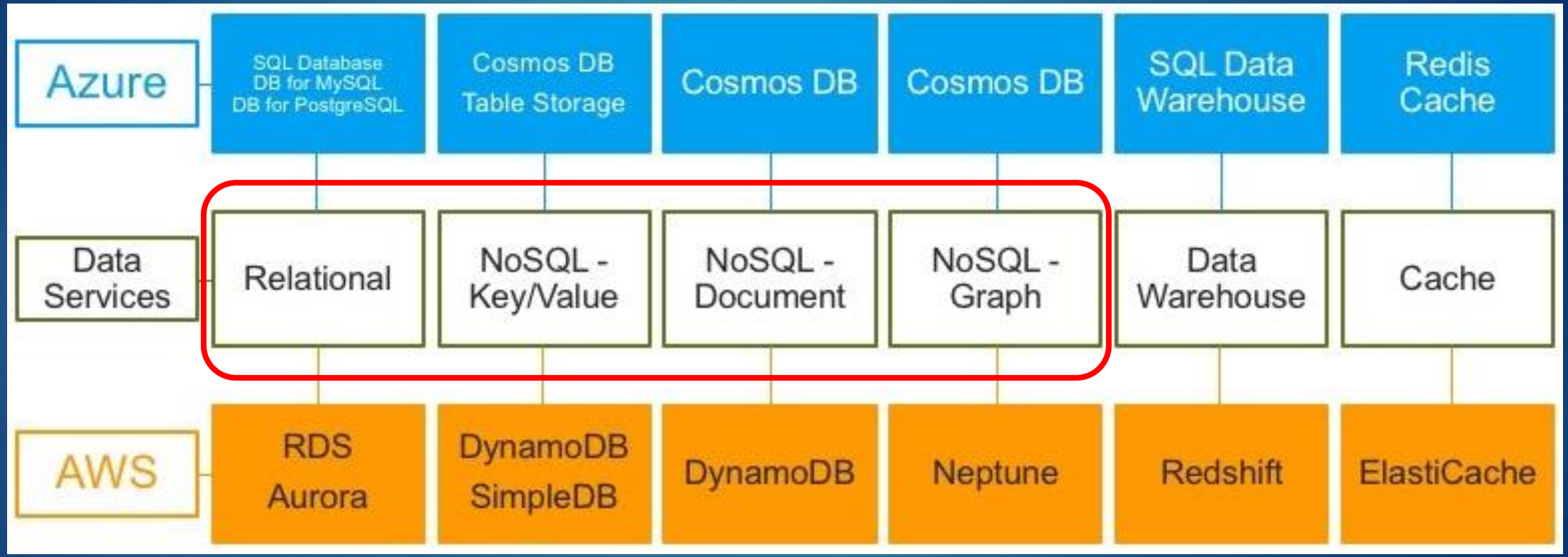
- **Handling a Large Number Of Read/Write Operations** – Use NoSQL databases when you need to scale fast. They can add nodes on the fly and can handle more concurrent traffic and large amounts of data with minimal latency.
- **Running data analytics** - NoSQL databases fit best for data analytics use cases, where you have to deal with an influx of massive amounts of data

Bottom-line - The choice between NoSQL and SQL depends on the complex business needs of an organization and volume and variety of data it consumes

Source: https://www.quora.com/Why-and-when-should-I-use-NoSQL-instead-of-SQL

What about other cloud providers?

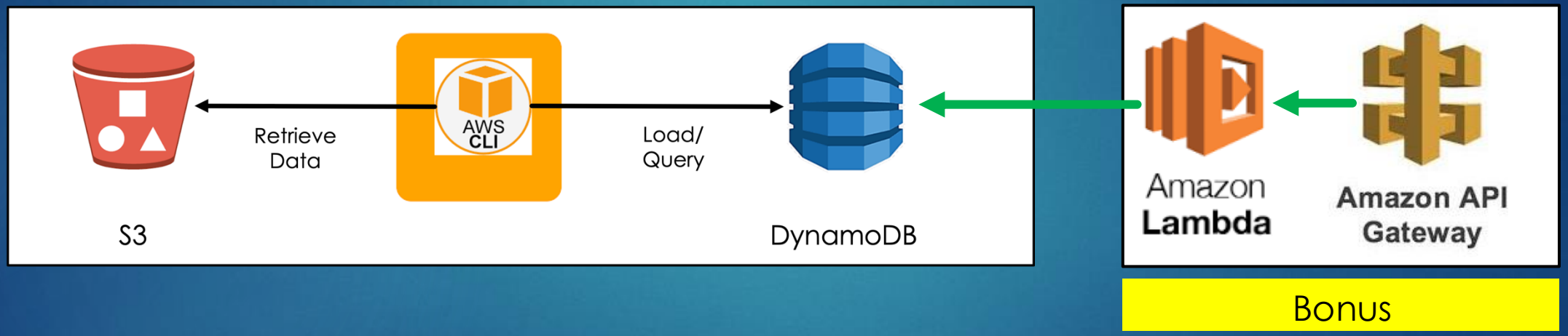
- AWS vs. Azure



Source: https://thomaslarock.com/2018/03/azure-versus-aws-data-services-comparison/

DynamoDB Activity

- You will be creating a DynamoDB instance and using the query and scan operations using AWS command-line interface (CLI)
- Data for the database will be transferred from an S3 bucket to your DynamoDB



- Located in Assignments > Activity - Create a DynamoDB
- There 1 deliverable plus an opportunity for a bonus point in this activity